

Ejercicios: Ordenamiento en Tiempo Lineal

Análisis y Diseño de Algoritmos

9 de junio de 2026

Ejercicio 1. A continuación se muestra el algoritmo de counting sort visto en clase. Modifique el algoritmo de modo que el nuevo algoritmo ordene de manera decreciente y que sea estable.

COUNTING-SORT(A, B, k)

```
1: Let  $C[0..k]$  be a new array
2: for  $i = 0$  to  $k$ 
3:    $C[i] = 0$ 
4: for  $j = 1$  to  $A.length$ 
5:    $C[A[j]] = C[A[j]] + 1$ 
6: // Now  $C[i]$  contains the number of elements equal to  $i$ 
7: for  $i = 1$  to  $k$ 
8:    $C[i] = C[i] + C[i - 1]$ 
9: // Now  $C[i]$  contains the number of elements  $\leq i$ 
10: for  $j = A.length$  downto 1
11:    $B[C[A[j]]] = A[j]$ 
12:    $C[A[j]] = C[A[j]] - 1$ 
```

Ejercicio 2. Diseñe un algoritmo que, dados $n^2 + n$ enteros en el rango de 0 a n^7 que devuelva el arreglo ordenado en forma no decreciente, con complejidad $O(n^2)$.

Ejercicio 3. Diseñe un algoritmo que recibe n enteros en el rango de -1 a $4n^2 + 4n - 1$ y retorna el arreglo ordenado. La complejidad debe ser $O(n)$

Ejercicio 4. Deseamos ordenar un arreglo de números enteros de 0 a $n^8 - 1$ utilizando radix sort. Determine si es más eficiente convertir a los números a base n o a base n^2 .

Ejercicio 5. Use inducción para demostrar que Radix Sort funciona correctamente. ¿En qué parte de su demostración se necesita la suposición de que el algoritmo de ordenamiento intermedio es estable?

Ejercicio 6. Dados n^2 números uniformemente distribuidos en $[0, 1[$, considere la siguiente variación de Bucket Sort. Asignaremos a cada número a uno de n buckets de modo que cada bucket corresponda a un subintervalo de $[0, 1[$ de igual longitud. Luego realizamos insertion sort en cada uno de los buckets y finalmente concatenamos las listas en orden. ¿Cuál es su complejidad esperada?

Ejercicio 7. Ilustre la operación del algoritmo Bucket-sort en el arreglo

$$A = [0,79, 0,13, 0,16, 0,64, 0,39, 0,20, 0,89, 0,53, 0,71, 0,42].$$

Ejercicio 8. Escriba el pseudocódigo del algoritmo de Bucket sort. El input es un arreglo de n números en el intervalo $[4, 12[$.

Ejercicio 9. Dados n puntos en el disco unitario, $p_i = (x_i, y_i)$, tales que $0 < x_i^2 + y_i^2 \leq 1$, para $i = 1, 2, \dots, n$. Suponga que los puntos están distribuidos uniformemente; es decir, la probabilidad de encontrar un punto en cualquier región del disco es proporcional al área de dicha región. Diseñe un algoritmo con tiempo de ejecución promedio $\Theta(n)$ para ordenar los n puntos según sus distancias al origen $d_i = \sqrt{x_i^2 + y_i^2}$.

(Sugerencia: Diseñe los tamaños de los cubos (buckets) en BUCKET-SORT de manera que reflejen la distribución uniforme de los puntos en el disco unitario.)

Ejercicio 10. Dado un arreglo de enteros `nums`, devuelva la diferencia máxima entre dos elementos consecutivos en su forma ordenada. Si el arreglo contiene menos de dos elementos, devuelva 0. Debe diseñar un algoritmo que se ejecute en tiempo lineal y utilice espacio extra lineal.